

Linux embarqué : Processus, Signaux et Threads

ISEN Nantes

M1 – Tronc commun

Audrey Queudet

audrey.queudet@isen-ouest.yncrea.fr

2025-2026

PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

PROCESSUS : DÉFINITION

- Un processus est un programme **en cours d'exécution**



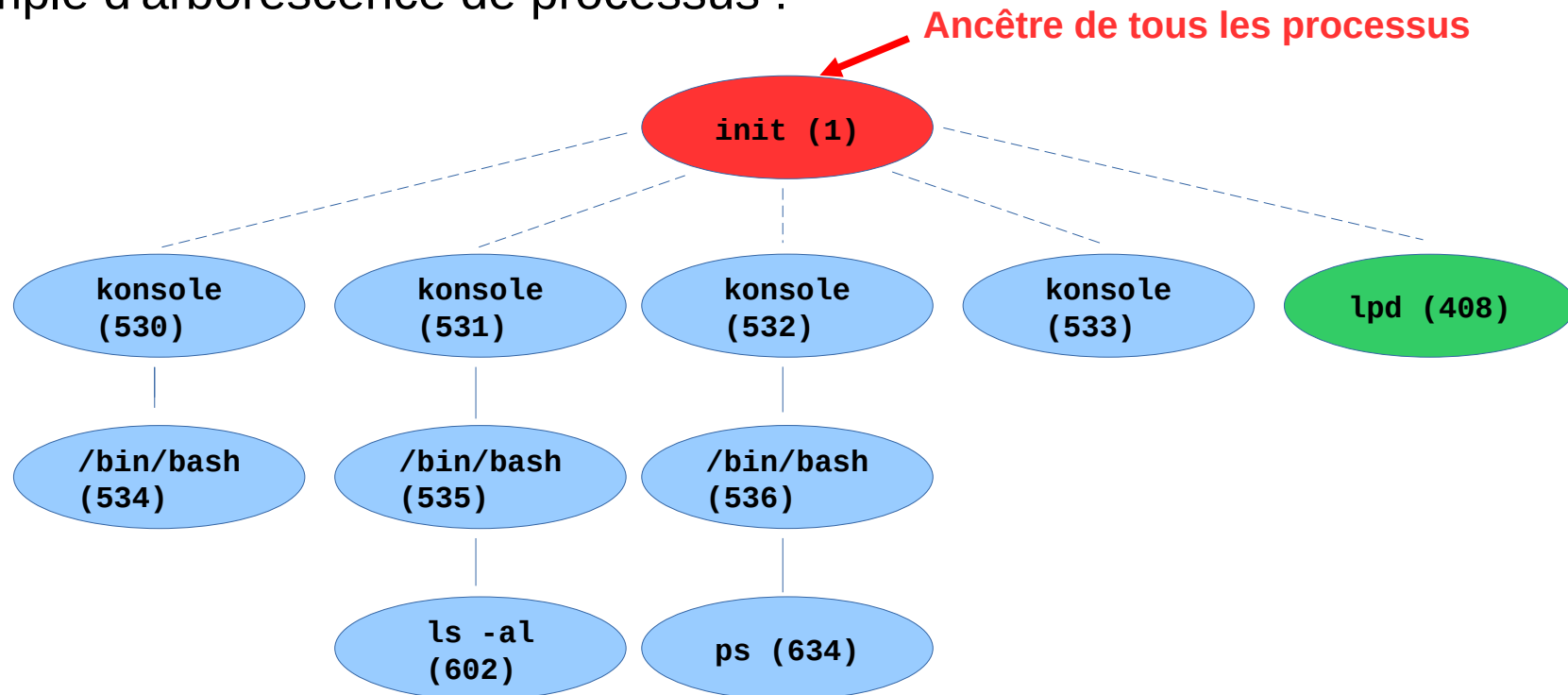
- Programme (= instructions machine) : entité statique
- Processus (= réalisation d'actions) : entité dynamique

- Un **processus** correspond à un espace indépendant de mémoire virtuelle auquel sont associés plusieurs éléments :

- son code
- ses données
- les ressources qui lui sont attribuées

TYPE DE PROCESSUS UNIX

- **Processus utilisateur** vs. **Processus système** (démons)
- Un processus est créé par un autre processus
- Exemple d'arborescence de processus :



CARACTÉRISTIQUES D'UN PROCESSUS UNIX

- Un processus possède de nombreuses caractéristiques :
 - ➔ le programme qu'il exécute (**CMD**)
 - ➔ un numéro d'identification que lui affecte le système (**PID**)
 - ➔ un créateur (**PPID**)
 - ➔ l'utilisateur pour lequel il fonctionne (**UID**)
 - ➔ le terminal ou la fenêtre du processus (**TTY**)
 - ➔ une consommation CPU (**CPU**)
 - ➔ une consommation mémoire (**MEM**)
 - ➔ une durée de traitement (**TIME**)
 - ➔ une heure de lancement (**STIME**)
 - ➔ un facteur de priorité (**C**)...

FICHIERS RELATIFS AUX PROCESSUS

- `/proc/$PID/` contient des informations sur le processus de PID `$PID`
 - ➔ **cmdline** : texte de la ligne de commande ayant lancé le processus
 - ➔ **status** : contient des informations sur l'état du processus
 - ➔ **statm** : informations sur l'utilisation de la mémoire
 - ➔ **io** : statistiques d'entrées et sorties pour le processus
 - ➔ **exe** : lien vers le fichier exécutable utilisé par le processus

COMMANDES UNIX DE GESTION DES PROCESSUS (1)

- Liste des processus satisfaisant un critère donné (spécifié en option) :

ps [-options]

- Exemples :

```
ps -ef
UID      PID  PPID  C  STIME     TTY     TIME    COMMAND
root      1    0    0  Dec 6     ?       1:02    init
...
jean     319  300    0 10:30:30   ?       0:02    /usr/dt/bin/dtsession
olivier  321  319    0 10:30:34  ttyt1    0:02    csh
olivier  324  321    0 10:32:12  ttyt1    0:00    ps -ef

ps -efu Olivier
olivier  321  319    0 10:30:34  ttyt1    0:02    csh
olivier  324  321    0 10:32:12  ttyt1    0:00    ps -efu
```

COMMANDES UNIX DE GESTION DES PROCESSUS (2)

- Visualisation de l'arbre des processus :

```
pstree [options] [pid, user]  
pstree -s 8744
```

- Affichage de la liste des processus de manière dynamique :

```
top -d secs -n max -u|U user -p pid -o fld -w [cols]
```

```
top - 21:02:16 up 13 min, 1 user, load average: 0,49, 0,62, 0,46
Tâches: 361 total, 1 en cours, 360 en veille, 0 arrêté, 0 zombie
%Cpu(s): 0,9 ut, 0,2 sy, 0,0 ni, 98,9 id, 0,1 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 31819,2 total, 26054,9 libr, 1843,3 util, 3920,9 tamp/cache
MiB Éch: 2048,0 total, 2048,0 libr, 0,0 util. 28523,4 dispo Mem
```

PID	UTIL.	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPS+	COM.
2956	queudet+	20	0	5045796	318192	116624	S	15,3	1,0	0:49.00	gnome-s+
2816	queudet+	20	0	995084	128060	83968	S	3,0	0,4	1:05.60	Xorg
8077	queudet+	20	0	1464804	388600	157920	S	0,7	1,2	1:27.06	soffice+
14	root	20	0	0	0	0	I	0,3	0,0	0:00.85	rcu_sch+
70	root	20	0	0	0	0	S	0,3	0,0	0:00.25	ksoftir+
71	root	20	0	0	0	0	I	0,3	0,0	0:02.57	kworker+
218	root	0	-20	0	0	0	D	0,3	0,0	0:01.04	kworker+
1645	root	20	0	340036	21156	17344	S	0,3	0,1	0:00.86	Network+
1678	root	20	0	276504	10768	9700	S	0,3	0,0	0:01.68	thermald
8523	queudet+	20	0	14792	4184	3376	R	0,3	0,0	0:00.09	top
1	root	20	0	168872	12260	8468	S	0,0	0,0	0:01.45	systemd

COMMANDES UNIX DE GESTION DES PROCESSUS (3)

- Changement de la priorité d'un processus :

```
nice -n valeur commande  
nice -n 5 gcc programme.c
```

- Envoi d'un signal à un processus :

```
kill [options] no_processus  
kill -9 521
```

- Lancement en arrière-plan d'un processus :

```
nom_processus &  
prog1 &
```

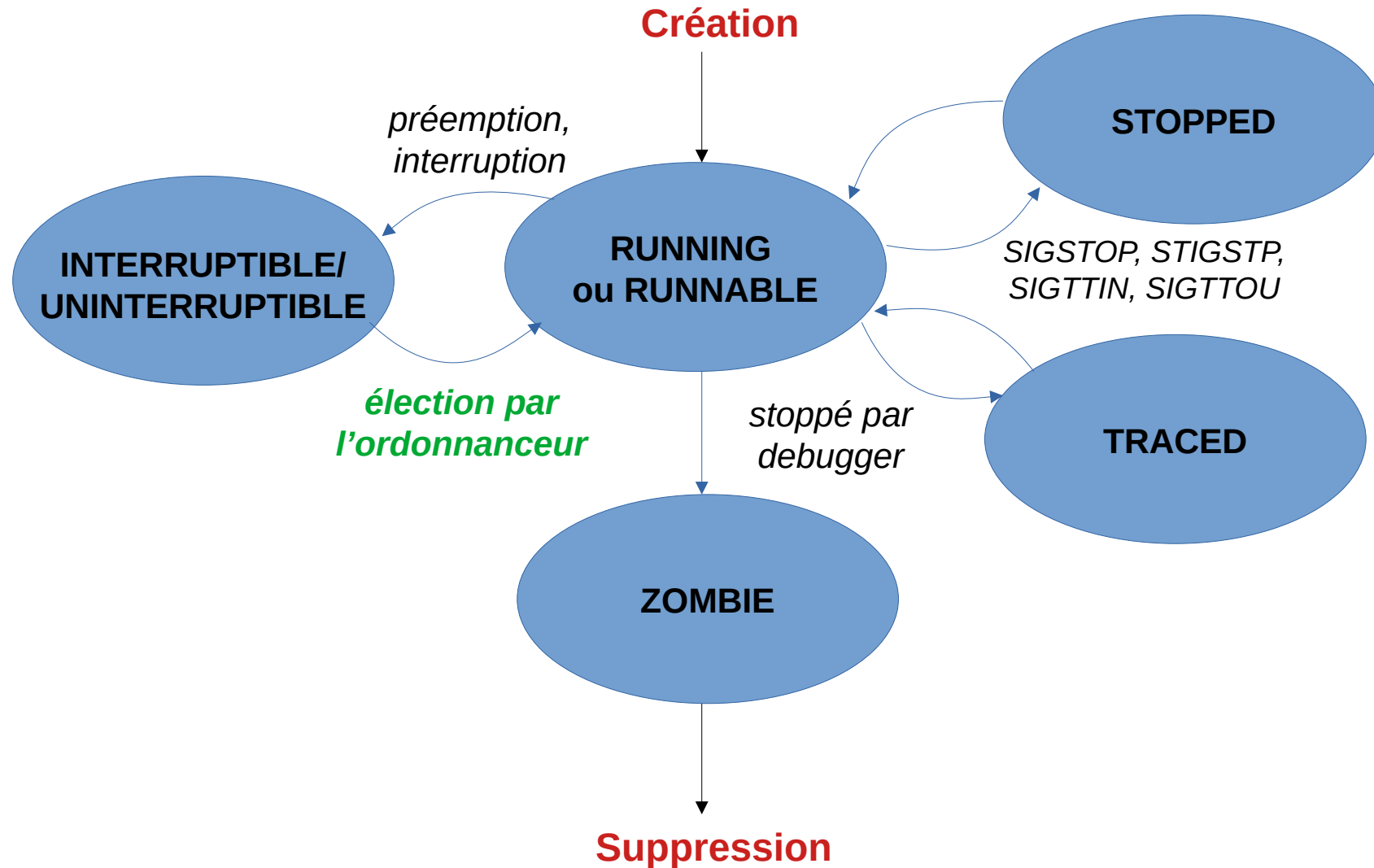
- Contrôle/Affichage de l'affinité des processus :

```
taskset [options] [mask | list ] [pid | command [arg]...]  
taskset -c 0,2,4-6 firefox  
taskset -cp 9067
```

ETATS DES PROCESSUS

- Flags mutuellement exclusifs
 - **RUNNING ou RUNNABLE (R)** : processus en cours d'exécution ou en attente du processeur
 - **INTERRUPTIBLE (I)** : processus suspendu en attente d'un événement (interruption HW, ressource système, signal, etc.)
 - **UNINTERRUPTIBLE (U)** : idem I sauf qu'un signal ne peut pas le réveiller
 - **STOPPED (S)** : processus arrêté (réception d'un signal SIGSTOP, SIGSTP, SIGTTIN, SIGTTOU)
 - **TRACED (T)** : processus arrêté par un debugger
 - **ZOMBIE (Z)** : processus terminé, Son père n'a pas encore effectué de wait() et pourrait avoir besoin de ces informations

VISUALISATION DES ETATS DES PROCESSUS

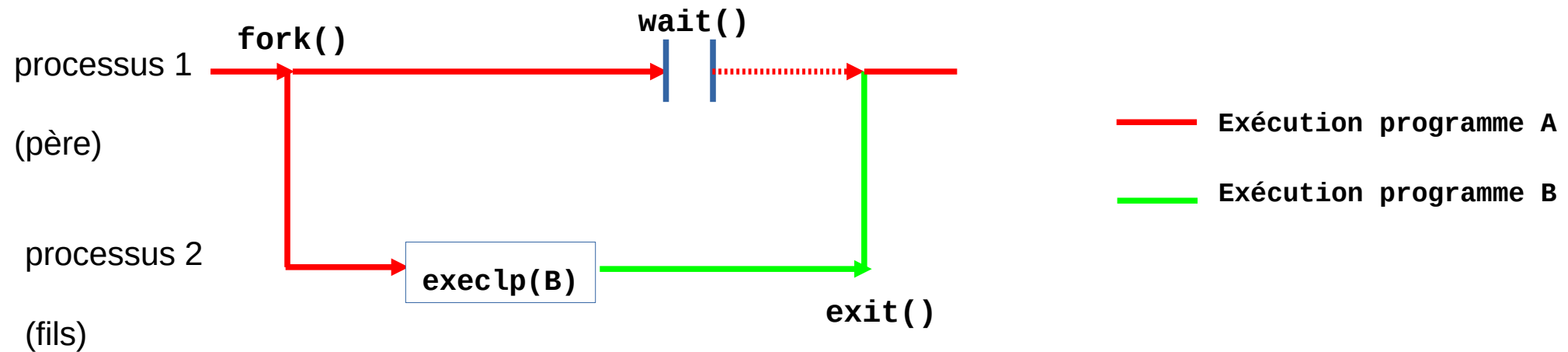


PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

MANIPULATION DES PROCESSUS EN LANGAGE C (1)

- Principes généraux :



- Valeurs de retour de `fork()`

- En cas de **succès** :
 - Le PID du fils est renvoyé au père
 - La valeur 0 est renvoyée au fils nouvellement créé
- En cas d'**échec** :
 - La valeur -1 est renvoyée au père

EXEMPLE DE DUPLICATION DE PROCESSUS

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    /*Création du fils*/
    pid = fork();
    if (pid == -1)
    {
        perror("fork");
        exit(1);
    }
    if (pid == 0)
        printf("Hello, je suis le fils !\n");
    else
        printf("Hello, je suis le père !\n");

    printf("Fin du programme.\n") ;
    return 0 ;
}
```

Code exécuté par le fils

Code exécuté par le père

Code exécuté par le père ET le fils

MANIPULATION DES PROCESSUS EN LANGAGE C (2)

- Primitives de gestion de processus :

```
pid_t fork ();
int execlp (char *comm, char *arg0, ..., NULL);
void exit (int status);
int wait (int *ptr_status);
pid_t waitpid(pid_t pid, int *status, int opts)
int sleep (int seconds);
int getpid ();
int getppid ();
int kill (pid_t pid, int sig);
int sched_[set|get]affinity(pid_t pid, size_t cpusetsize, cpu_set_t *mask);
```

PLAN DU COURS

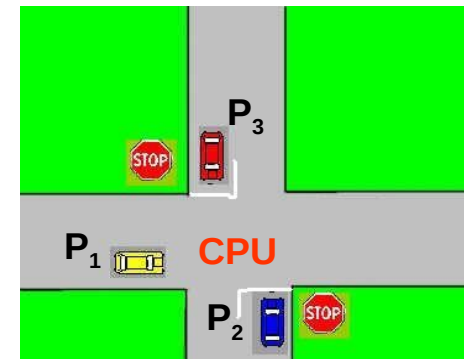
- Les processus UNIX
- La gestion des processus en C
- **L'ordonnancement des processus**
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

ALLOCATION DU PROCESSEUR AUX PROCESSUS

- Comment contrôler l'ordre de passage des processus sur le processeur ?
- Comment contrôler la répartition du temps d'exécution entre les processus ?



L'ordonnanceur



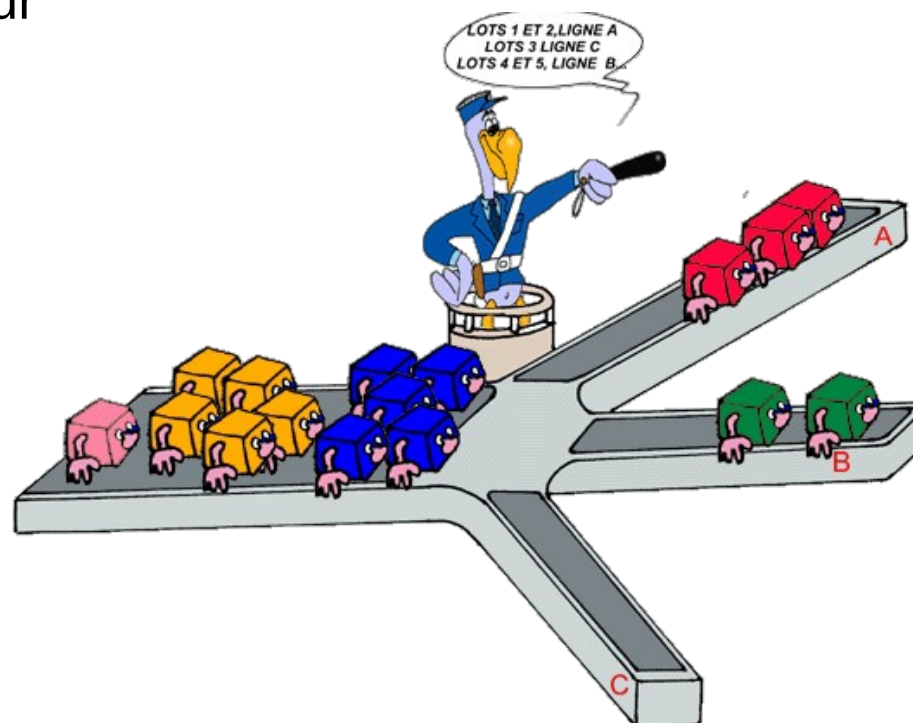
- L'ordonnanceur est un composant (procédure) du système d'exploitation

ORDONNANCEMENT : QUELS OBJECTIFS ?

- L'ordonnancement consiste à :
 - **choisir** le processus à exécuter à un instant t
 - **déterminer** le temps durant lequel le processeur lui sera alloué
- **Objectif d'ordonnancement :**
 - Maximiser le nombre de processus exécutés par unité de temps
 - Minimiser le temps d'attente d'exécution de chaque processus
 - Maximiser le taux d'utilisation des processeurs et autres ressources
 - Favoriser les processus les plus prioritaires
 - Minimiser le nombre et la durée des changements de contexte

TYPOLOGIE DES ALGORITHMES D'ORDONNANCEMENT

- Monoprocasseur / multiprocasseur
- En-ligne / Hors-ligne
- Préemptif / Non préemptif
- Oisif / Non oisif
- Centralisé / Réparti



ORDONNANCEMENT NON PREEMPTIF

- Ordonnancement **selon l'ordre d'arrivée**
 - premier arrivé, premier servi

First Come First Served (FCFS)



- Ordonnancement **selon la durée de calcul**
 - travail le plus court d'abord

Shortest Job First (SJF)

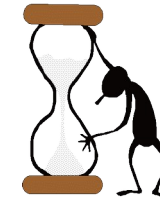


ORDONNANCEMENT PREEMPTIF

- Ordonnancement **selon la durée de calcul restante**

→ temps restant le plus court d'abord

Shortest Remaining Time (SRT)

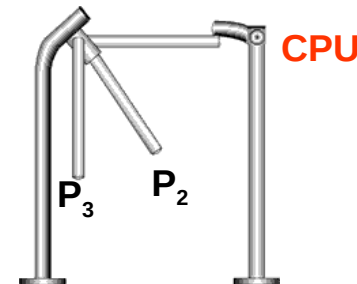


P_1

- Ordonnancement **sans notion de priorité**

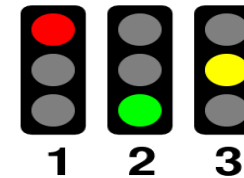
→ temps-partagé avec politique du tourniquet

Round-Robin (RR)



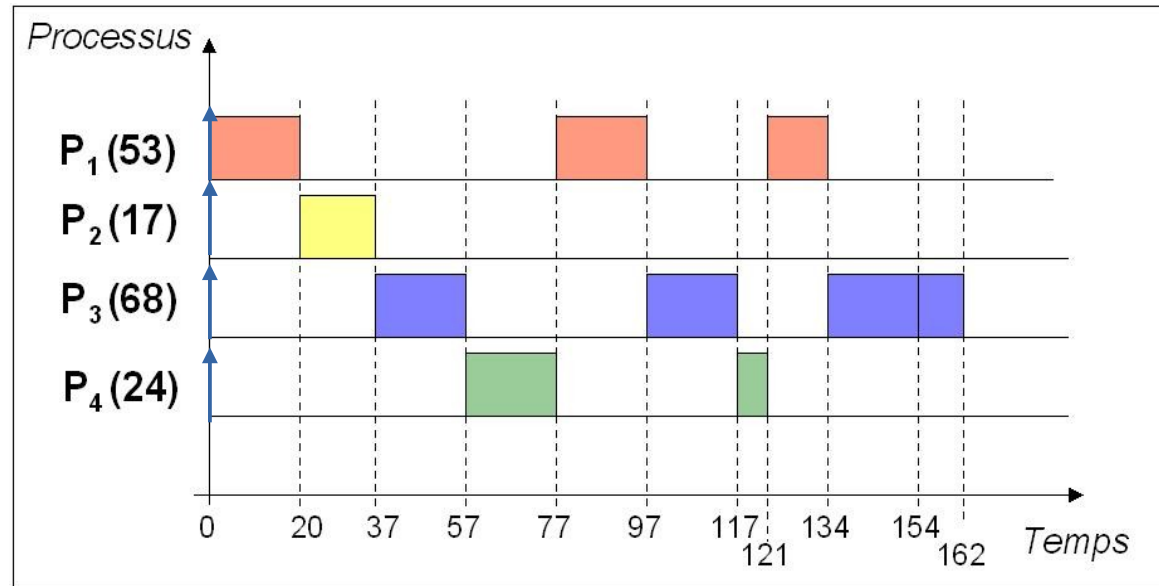
- Ordonnancement **à priorités** (statiques ou dynamiques)

→ la tâche la plus prioritaire obtient le processeur



ORDONNANCEMENT TEMPS PARTAGÉ (ROUND-ROBIN)

- **Principe** : allocation du processeur par tranche (quantum) de temps
- **Exemple** ($q=20$, $n=4$) :



- Chaque processus obtient le processeur au bout de $(n-1)*q$ unités de temps au plus

ORDONNANCEMENT TEMPS PARTAGÉ (ROUND-ROBIN)

- **Intérêts :** 😊

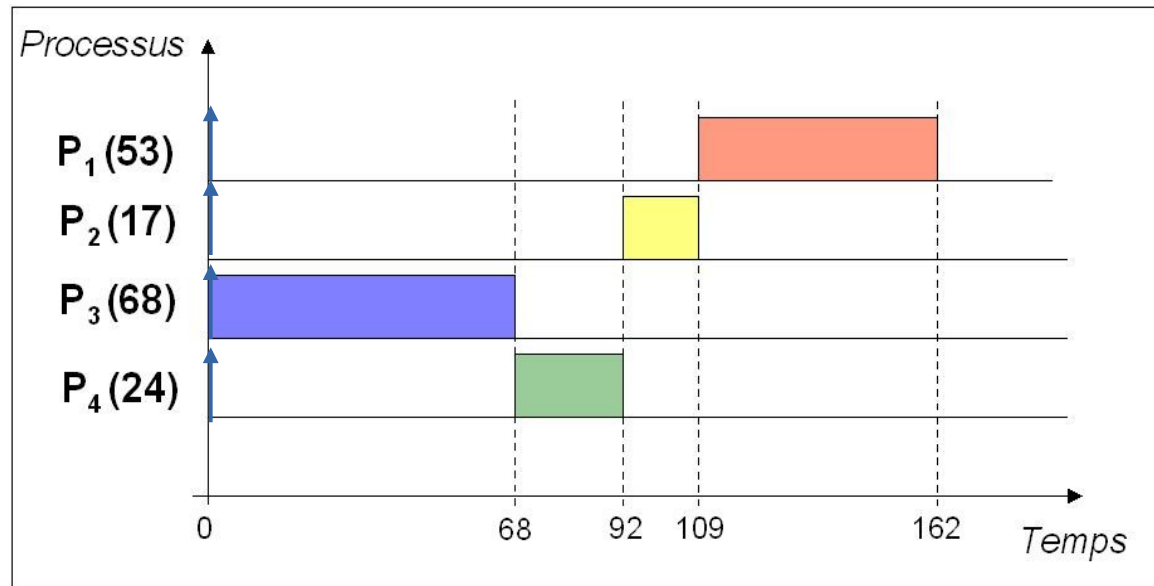
- Équité de l'attribution du processeur entre tous les processus
- Mise en œuvre simple

- **Inconvénients :** 😞

- Pas de prise en compte de l'importance relative des processus
- Difficulté du choix de la tranche de temps
 - ♦ Si q est trop grand, Round-Robin devient équivalent à FCFS
 - ♦ Si q est trop petit, il y a augmentation du nombre de changements de contexte !

ORDONNANCEMENT A PRIORITÉS STATIQUES

- **Principe** : allocation du processeur selon des priorités statiques (numéros affectés aux processus pour toute la vie de l'application)
- **Exemple** ($\text{priorités}(P_1, P_2, P_3, P_4)=(3,2,0,1)$) :



- Dans certains systèmes, l'échelle des priorités est inversée (0 est alors la priorité la + faible)

ORDONNANCEMENT A PRIORITÉS STATIQUES

- **Intérêts :** 😊

- ➔ Prise en compte de l'importance relative des processus
- ➔ Mise en œuvre simple

- **Inconvénients :** 😞

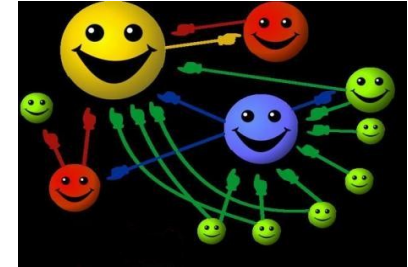
- ➔ Problèmes de blocages durant des périodes de temps illimitées
- ➔ Problèmes de famines des processus de moindres priorités

PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- **La communication par tubes**
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

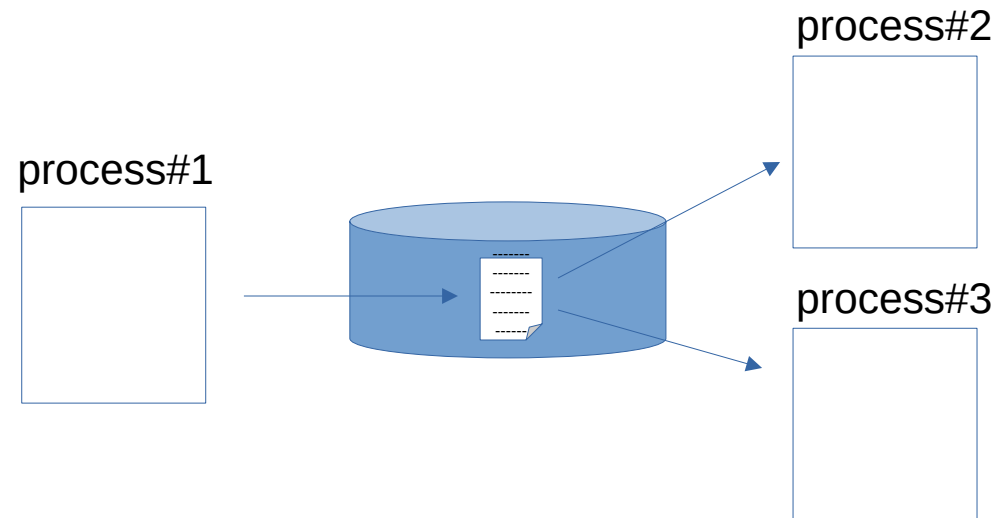
COMMUNICATION INTER-PROCESSUS

- **Inter-Process Communication (IPC)** : méthodes permettant à plusieurs processus de communiquer entre eux
- 3 catégories de mécanismes :
 - ➔ outils permettant aux processus de **s'échanger des données**
 - ◆ les fichiers
 - ◆ la mémoire partagée
 - ➔ outils permettant de **synchroniser des processus**
 - ◆ les sémaphores
 - ◆ les signaux
 - ➔ outils permettant d'**échanger des données et de synchroniser des processus**
 - ◆ les tubes (pipes)
 - ◆ les files d'attente de message



COMMUNICATION PAR FICHIER

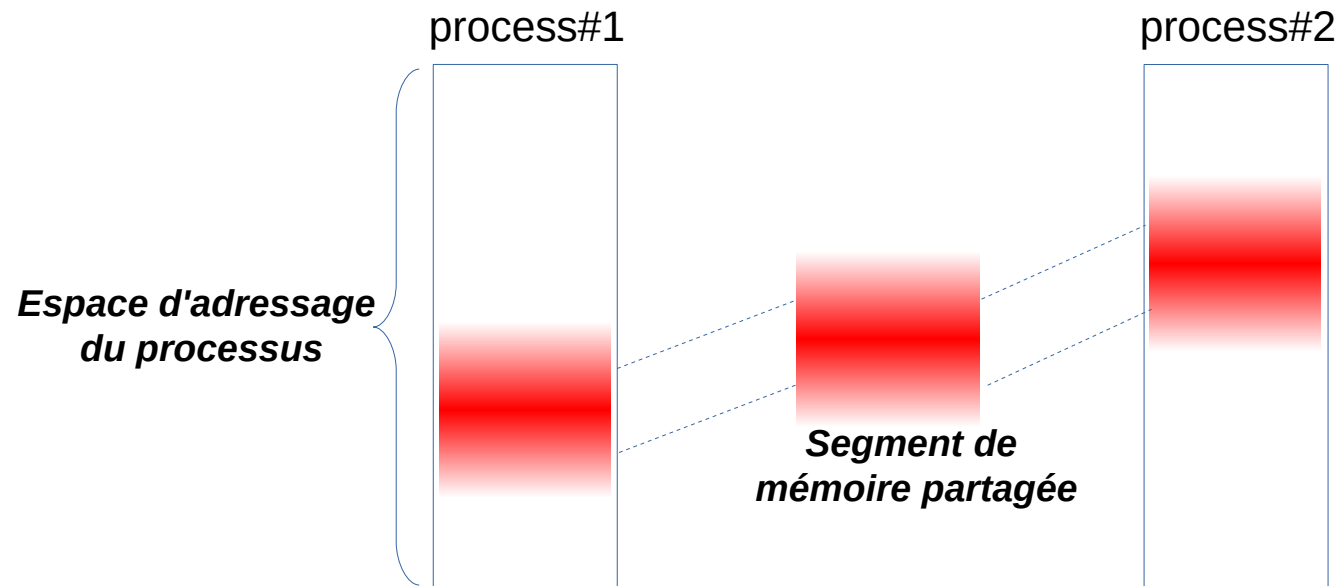
- Les fichiers constituent la manière primitive de communication :



- Approche **coûteuse** (accès au système de fichier du disque)
- Afin d'éviter les accès concurrents en lecture / écriture, on procède à des verrouillages

COMMUNICATION PAR MÉMOIRE PARTAGÉE

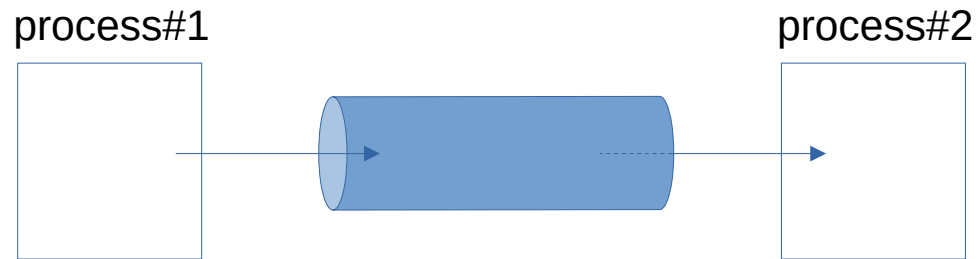
- Un segment de mémoire peut être simultanément attaché à l'espace virtuel de plusieurs processus, ou plusieurs fois à des adresses différentes d'un même processus



- Afin d'éviter les accès concurrents en lecture / écriture, on procède à des verrouillages

COMMUNICATION PAR TUBE (1)

- Un tube (pipe en anglais) est un **flux de données** permettant l'**échange unidirectionnel** de données entre deux processus s'exécutant sur une même machine



- Les tubes peuvent être considérés comme des fichiers ouverts (sans l'inconvénient des accès coûteux au système de fichier du disque)
- Utilisation simple en **langage shell** (opérateur '|')

```
ls | more
```

COMMUNICATION PAR TUBE (2)

- Tubes anonymes entre processus à lien de parenté (<unistd.h>) :

```
int pipe(int pipefd[2]);
```

création d'un tube anonyme

```
ssize_t read(int fd, void *buf, size_t count);
```

lecture dans un tube anonyme

```
ssize_t write(int fd, const void *buf, size_t count);
```

écriture dans un tube anonyme

```
int close(int fd);
```

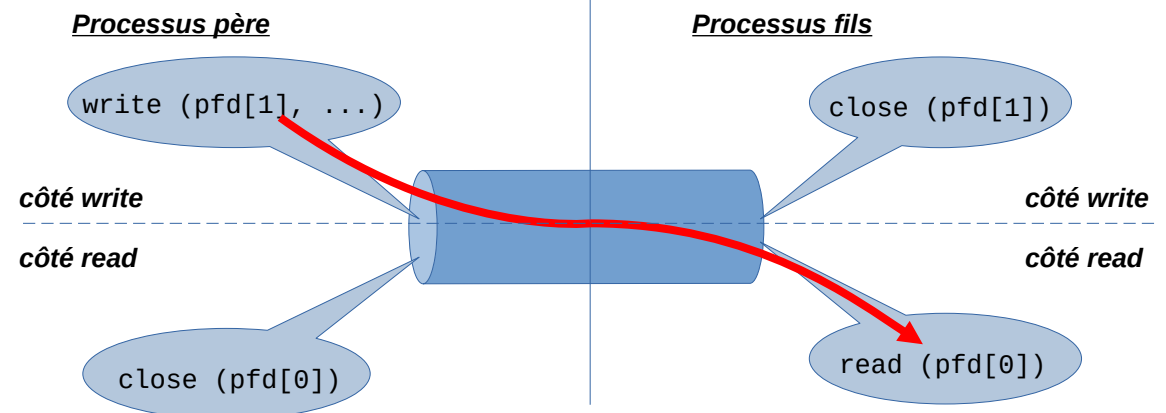
fermeture d'un tube anonyme

EXEMPLE DE COMMUNICATION PAR TUBE

```
int main()
{
    char buffer[BUFSIZ];
    int pfd[2];
    if (pipe(pfd) == -1)
    {
        printf("pipe failed\n");
        return 1;
    }

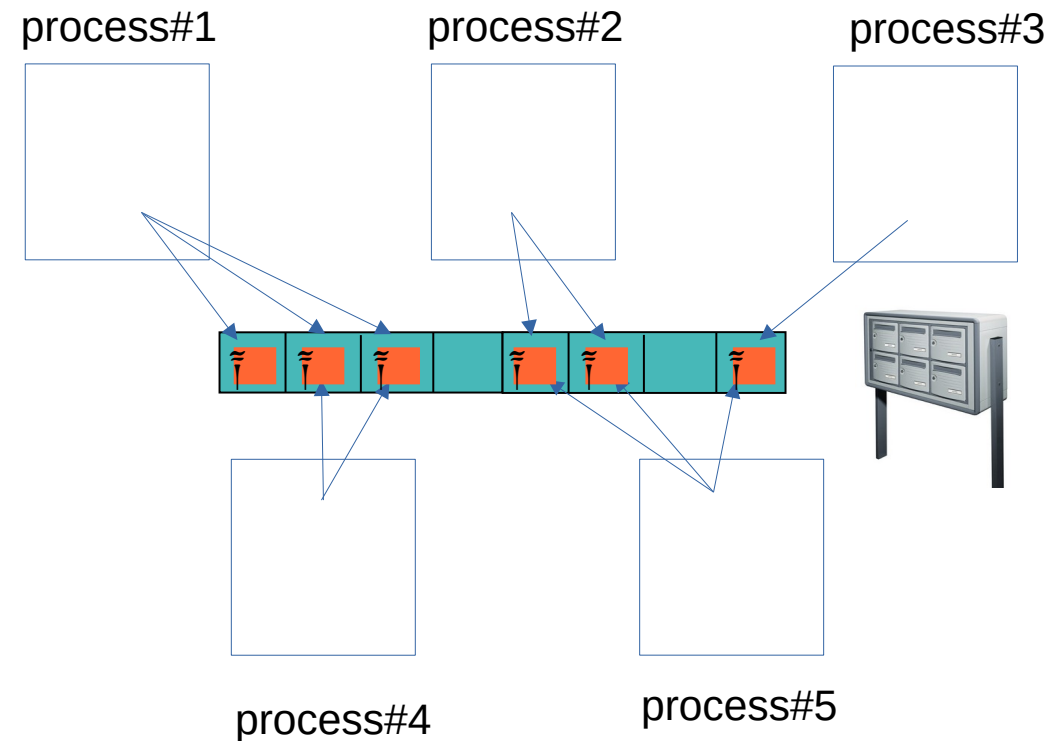
    /* create the child */
    int pid;
    ...
    if (pid == 0)
    {
        /*Lecture du fils sur le tube pf*/
        close(pfd[1]);/* fermeture côté write */
        /* Lecture des données */
        while (read(pfd[0], buffer, BUFSIZ) != 0)
            printf("child reads %s", buffer);
        close(pfd[0]);/* fermeture côté read */
    }
}
```

```
else
{
    /*Ecriture du père sur le tube pf*/
    close(pfd[0]); /* fermeture côté read */
    /* Envoi des données */
    strcpy(buffer, "Salut mon fils !\n");
    write(pfd[1], buffer, strlen(buffer)+1);
    close(pfd[1]); /* fermeture côté write */
    wait(NULL);
}
return 0 ;
} // fin du main()
```



COMMUNICATION PAR FILE DE MESSAGES

- Les files de messages permettent une **communication indirecte** entre processus (mécanisme de « **boîte aux lettres** »)

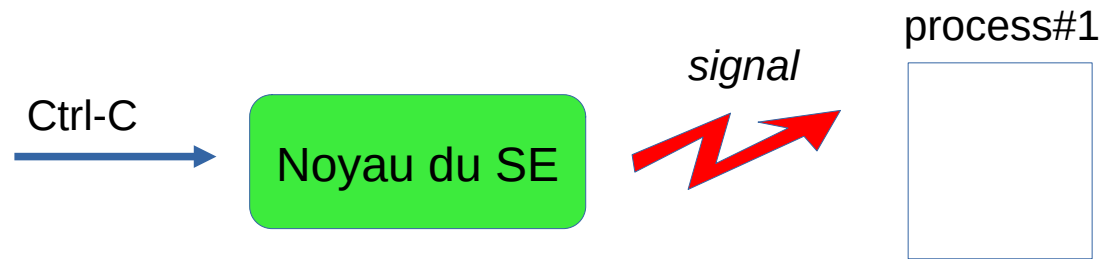


PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- **La synchronisation par signaux**
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

SYNCHRONISATION PAR SIGNAUX

- Les signaux permettent d'avertir simplement un processus de l'arrivée d'un **événement**



- Les signaux sont limités au rôle de notification d'événement

SIGNAUX STANDARDS

- Les signaux sont définis dans le fichier d'entête `signal.h`
- A chaque signal est associé un numéro (variable selon les architectures)

Signal	x86/ARM most others	Alpha/ SPARC	MIPS
SIGHUP	1	1	1
SIGINT	2	2	2
SIGQUIT	3	3	3
SIGILL	4	4	4
SIGTRAP	5	5	5
SIGABRT	6	6	6
SIGIOT	6	6	6
SIGBUS	7	10	10
SIGEMT	-	7	7
SIGFPE	8	8	8
SIGKILL	9	9	9
SIGUSR1	10	30	16
SIGSEGV	11	11	11
SIGUSR2	12	31	17
SIGPIPE	13	13	13
SIGALRM	14	14	14
SIGTERM	15	15	15
SIGSTKFLT	16	-	-
SIGCHLD	17	20	18
SIGCLD	-	-	18
SIGCONT	18	19	25
SIGSTOP	19	17	23
SIGTSTP	20	18	24
SIGTTIN	21	21	26
SIGTTOU	22	22	27
SIGURG	23	16	21
SIGXCPU	24	24	30
SIGXFSZ	25	25	31
SIGVTALRM	26	26	28
SIGPROF	27	27	29
SIGWINCH	28	28	20
SIGIO	29	23	22
SIGPOLL			
SIGPWR	30	29/-	19
SIGINFO	-	29/-	-
SIGLOST	-	-/29	-
SIGSYS	31	12	12
SIGUNUSED	31	-	-

```

queudet-a@dell-queudet-2021:~$ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT      4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH     29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX

```

ENVOI D'UN SIGNAL

- Via un caractère de contrôle :

CRTL-C	envoie SIGINT (interruption)
CRTL-Z	envoie SIGSTOP (suspension)
CRTL-\	envoie SIGQUIT (fin)

- En ligne de commande :

```
kill [-no_signal] no_pid
kill -9 4014

kill [-nom_signal] no_pid
kill -SIGUSR1 2584
```

- Via un appel de fonction :

```
int kill(pid_t pid, int sig);
```

COMPORTEMENT PAR DÉFAUT

- A chaque signal est associé un comportement par défaut

- Modification de ce comportement par défaut :

→ En ligne de commande :

```
trap 'instruction1; instruction2;...' nom_signal
trap 'echo fini;exit 1' SIGUSR1
```

→ Via un appel de fonction :

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

- Certains signaux sont non modifiables : SIGKILL, SIGSTOP, SIGCONT

POSIX signal	default action
SIGHUP	terminate
SIGINT	terminate
SIGQUIT	coredump
SIGILL	coredump
SIGTRAP	coredump
SIGABRT/SIGIOT	coredump
SIGBUS	coredump
SIGFPE	coredump
SIGKILL	terminate(+)
SIGUSR1	terminate
SIGSEGV	coredump
SIGUSR2	terminate
SIGPIPE	terminate
SIGALRM	terminate
SIGTERM	terminate
SIGCHLD	ignore
SIGCONT	ignore(*)
SIGSTOP	stop(*)(+)
SIGTSTP	stop(*)
SIGTTIN	stop(*)
SIGTTOU	stop(*)
SIGURG	ignore
SIGXCPU	coredump
SIGXFSZ	coredump
SIGVTALRM	terminate
SIGPROF	terminate
SIGPOLL/SIGIO	terminate
SIGSYS/SIGUNUSED	coredump
SIGSTKFLT	terminate
SIGWINCH	ignore
SIGPWR	terminate
SIGRTMIN-SIGRTMAX	terminate

EXAMPLE

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handINT (int signo){
    printf ("reception d'un signal SIGINT\n");
}

void main(){
    signal (SIGINT, handINT) ;
    printf("processus : %d\n", getpid());
    while (1) ;
}
```

```
processus : 21124
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
^Creception d'un signal SIGINT
```

DELAI PRÉPROGRAMMÉ

- Un signal peut être généré suite à l'expiration de délai préprogrammé :

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int seconds);
```

- Mise en marche d'une minuterie (secondes > 0)
- Valeur renvoyée : nb de secondes restantes avant l'arrêt de la minuterie
- Envoi du signal SIGALRM lorsque la minuterie expire
- Ne pas oublier de dérouter le signal SIGALRM (la terminaison est le comportement par défaut)



ATTENTE PASSIVE DE L'ARRIVÉE D'UN SIGNAL

- Un processus peut se mettre en attente de l'arrivée d'un signal quelconque :

```
#include <unistd.h>

int pause(void);
```

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handINT (int signo){
    printf ("reception d'un signal SIGINT\n");
}

void main(){
    signal (SIGINT, handINT) ;
    printf("processus : %d\n", getpid());
    pause() ;
}
```

- Un processus peut se mettre en attente de l'arrivée d'un signal particulier :

```
#include <signal.h>

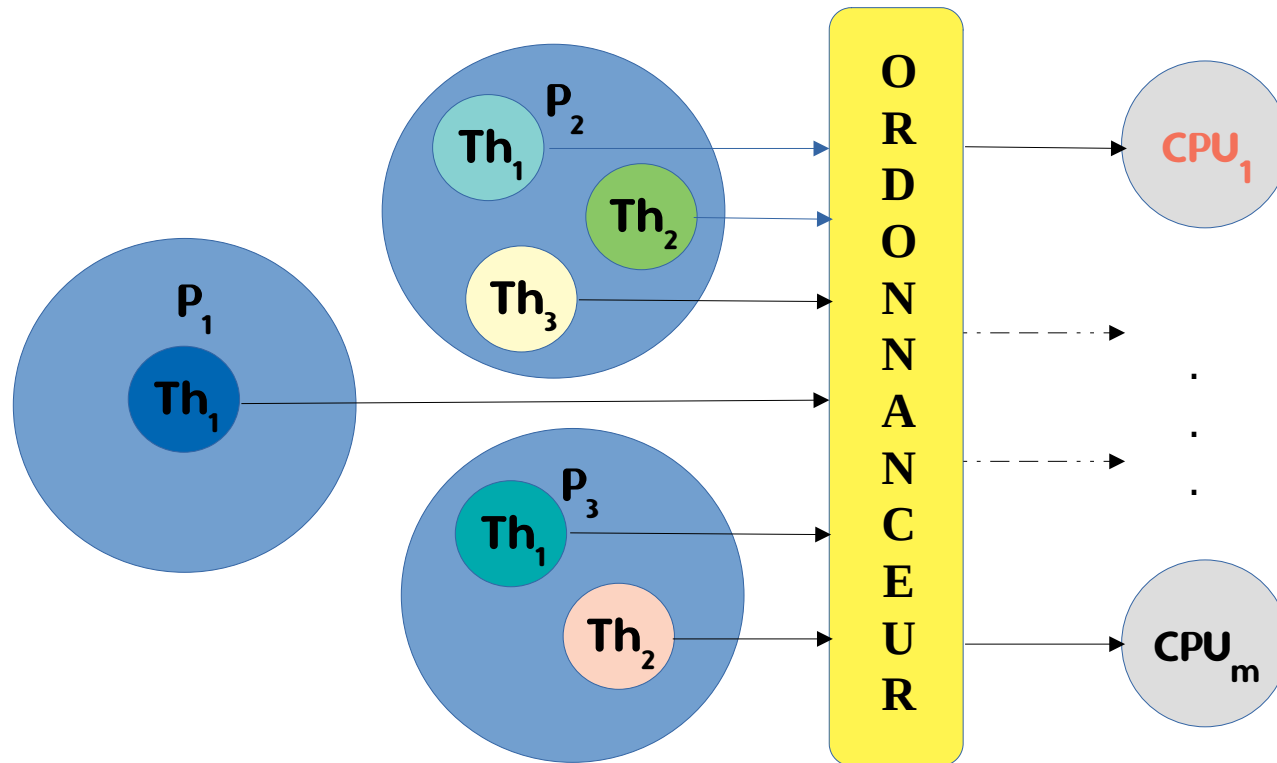
int sigsuspend(const sigset_t *mask);
```

PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- **Les threads**
- La gestion des threads en C
- La synchronisation par sémaphores

THREAD : DÉFINITION

- Un **thread** est un fil d'exécution dans le code du processus



THREAD : PRINCIPE

- Un **thread** est doté :
 - ➔ d'un compteur programme (suivi des instructions à exécuter)
 - ➔ de registres systèmes (variables de travail en cours)
 - ➔ d'une pile (historique de l'exécution)
- Plusieurs processus permettent à un ordinateur d'effectuer plusieurs tâches à la fois.
Ils se partagent les **ressources physiques**.
- Plusieurs threads permettent à un processus de décomposer le travail à exécuter en parallèle.
Ils se partagent les **ressources physiques et virtuelles**.

INTÉRÊTS DE LA PROGRAMMATION MULTI-THREAD

- **Traitements accélérés**
 - Multiplication de matrice
- **Débit plus élevé**
 - Un thread bloqué n'impacte pas la progression des autres threads
- **Réactivité accrue**
 - IHM utilisateur / Jeux plus fluides
- **Communication moins coûteuse**
 - Les threads d'un même processus partagent les variables globales

PLAN DU COURS

- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

MANIPULATION DES THREADS EN C

- Primitives de gestion des threads :

```
int pthread_create(    pthread_t *idptr,
                      pthread_attr_t attr,
                      void *(*fonc) (void *),
                      void *arg);
```

```
void pthread_exit( void *status );
```

```
int pthread_join( pthread_t id, void *status );
```

```
int pthread_kill(pthread_t thread, int sig);
```

```
int pthread_[set|get]affinity_np(pthread_t thread, size_t
cpusetsize, const cpu_set_t *cpuset);
```

EXEMPLE

```
#include <stdio.h>
#include <pthread.h>

void *hello( void *arg ) {
    int *id = (int*)arg;
    printf("%d: Salut ! \n", *id);
    pthread_exit(NULL);
}

int main () {
    pthread_t thread[3];
    int id[3]={1,2,3};
    int i;
    for (i=0;i<3;i++) {
        printf("Thread %d créé\n",i);
        pthread_create(&thread[i], NULL, hello, (void *)&id[i]);
    }
    for (i=0;i<3;i++)
        pthread_join(thread[i], NULL);
    return 0;
}
```

```
Thread 0 créé
Thread 1 créé
Thread 2 créé
2: Salut !
1: Salut !
3: Salut !
```

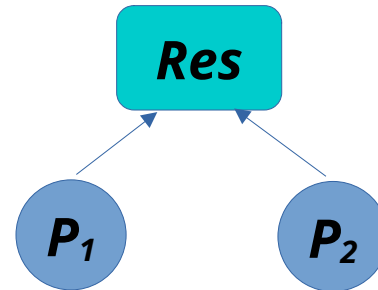
gcc votre_programme.c -lpthread -o votre_programme

PLAN DU COURS

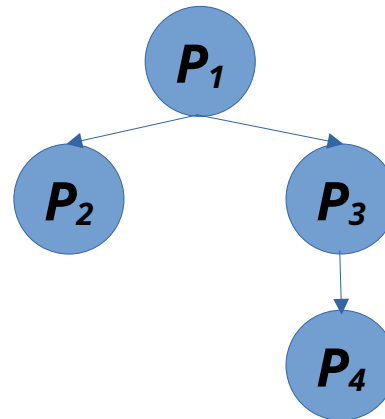
- Les processus UNIX
- La gestion des processus en C
- L'ordonnancement des processus
- La communication par tubes
- La synchronisation par signaux
- Les threads
- La gestion des threads en C
- La synchronisation par sémaphores

SYNCHRONISER LES PROCESSUS : DANS QUELS CAS ?

- Pour garantir la **cohérence des ressources partagées**



- Pour contrôler les **dépendances** d'exécution entre processus

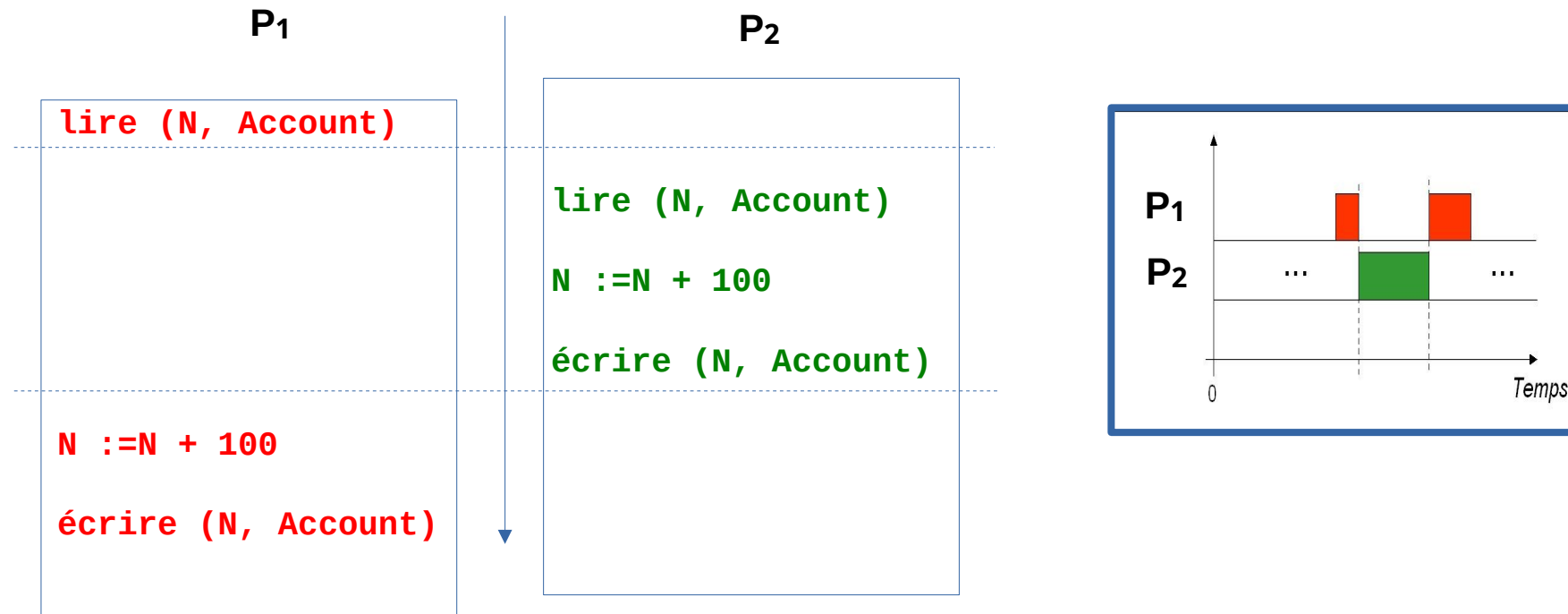


LE PROBLÈMES DES RESSOURCES PARTAGÉES

- Plusieurs processus → **accès concurrents** aux ressources
- Une **ressource** désigne toute entité dont a besoin un processus pour s'exécuter :
 - ressource **matérielle** (processeur, périphérique, etc.)
 - ressource **logicielle** (variable)
- Une ressource est dite **critique** lorsque des accès concurrents à cette ressource peuvent mener à un état incohérent
- Une **section critique** est une section de programme manipulant une ressource critique
- Un mécanisme d'**exclusion mutuelle** doit assurer l'atomicité des sections critiques relatives à une ressource critique

CAS D'INCOHÉRENCE DE DONNÉES

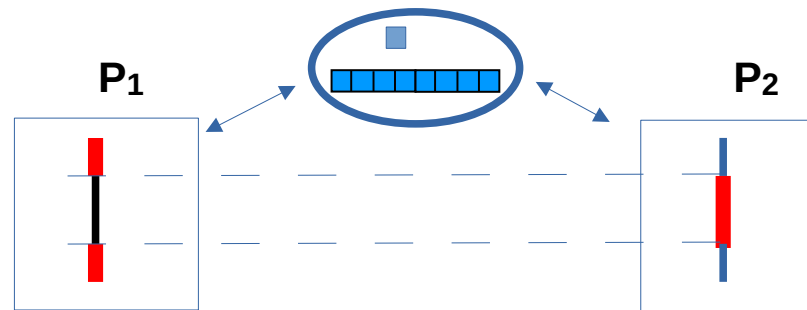
- Problème de la synchronisation relative de l'exécution des processus



→ P_2 ne doit pas accéder à $Account$ tant que P_1 l'utilise !

SOLUTION : LES SEMAPHORES D'EXCLUSION MUTUELLE

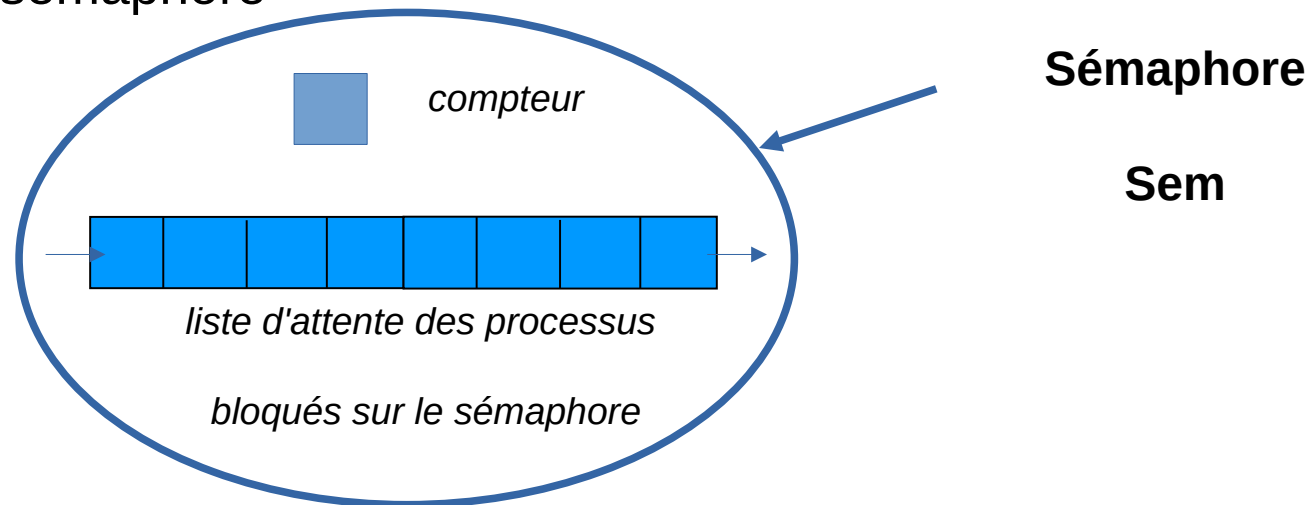
- Mise en œuvre de l'exclusion mutuelle



Section critique

PRINCIPE DES SEMAPHORES

- Un sémaphore est une **structure de données**
 - contenant un compteur (valeur entière non négative)
 - gérant une file d'attente de tâches attendant qu'advienne une condition particulière propre au sémaphore

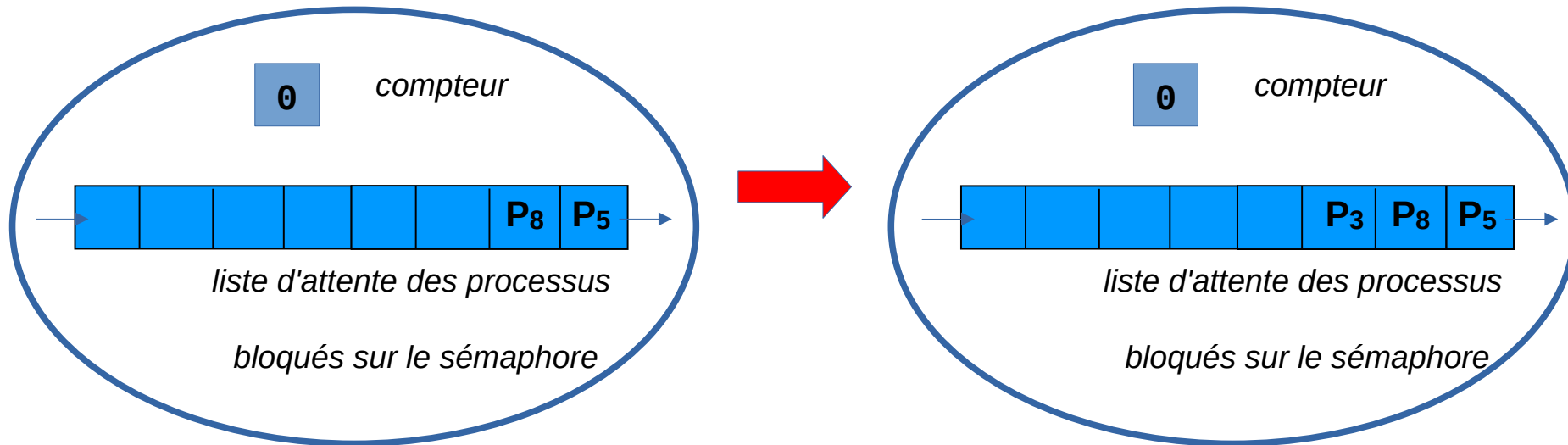
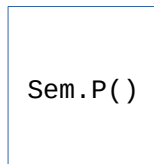


SEMAPHORES : OPÉRATIONS POSSIBLES (1)

- 1ère opération : **test de prise** du sémaphore (« Puis-je ? »)

Sem.P() : *si (Sem.compteur > 0)*
 alors Sem.compteur = Sem.compteur - 1
 sinon insère_ce_processus(Sem.file)
 finsi

P₃

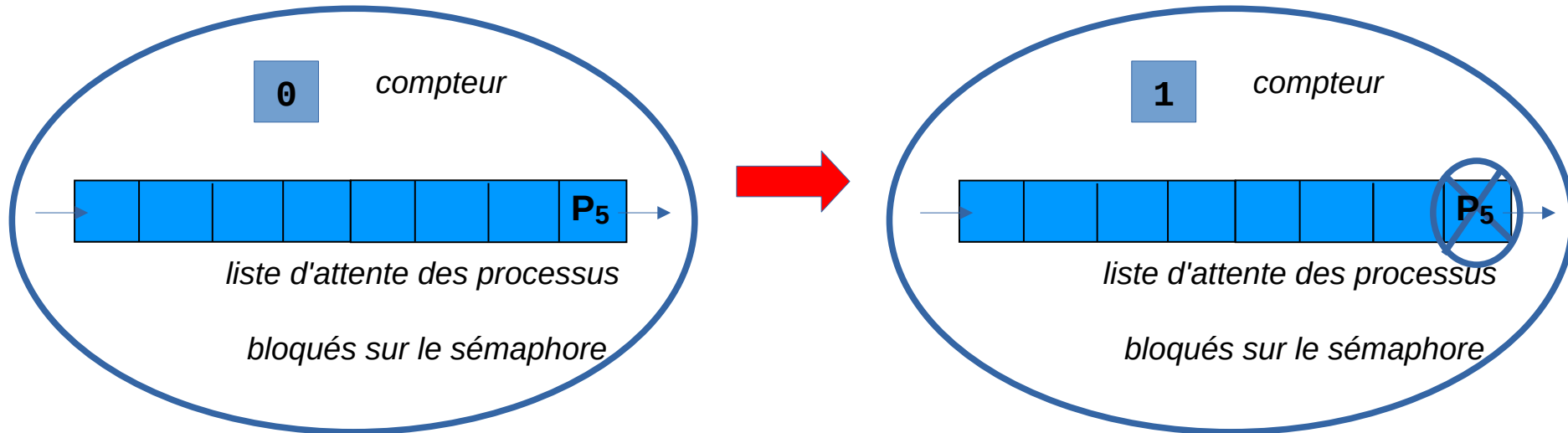
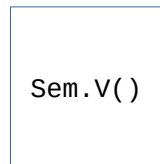


SEMAPHORES : OPÉRATIONS POSSIBLES (2)

- 2ème opération : **libération** du sémaphore (« Vas-y ! »)

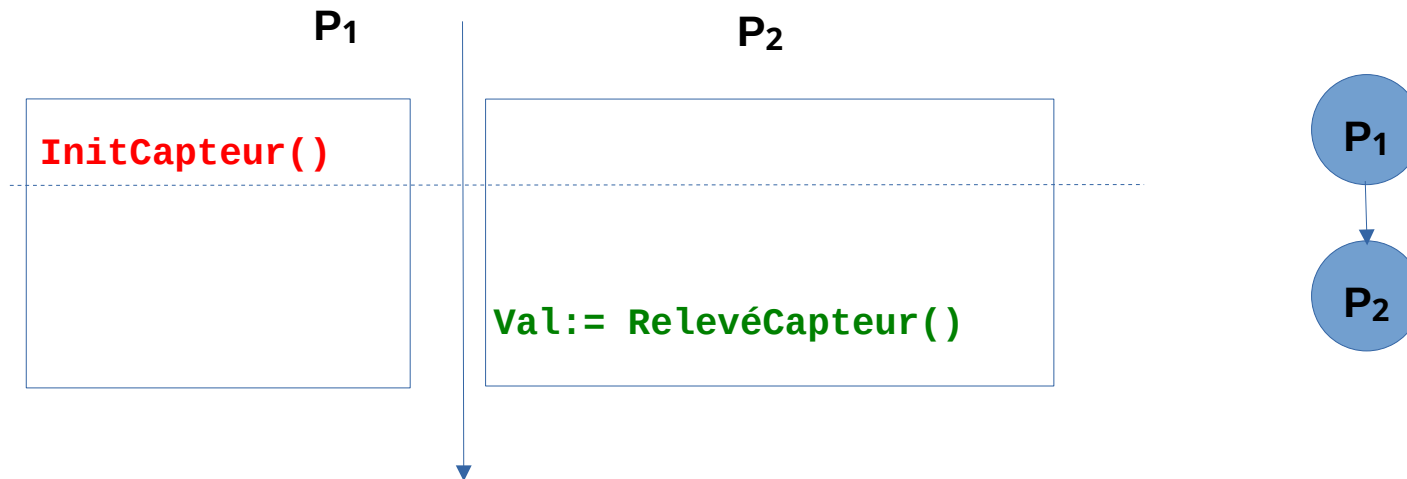
```
Sem.V() : Sem.compteur = Sem.compteur+1
          si (Sem.compteur > 0)
            alors extrait_un_processus(Sem.file)
          fin si
```

P₃



CONTRÔLER LES DÉPENDANCES D'EXÉCUTION

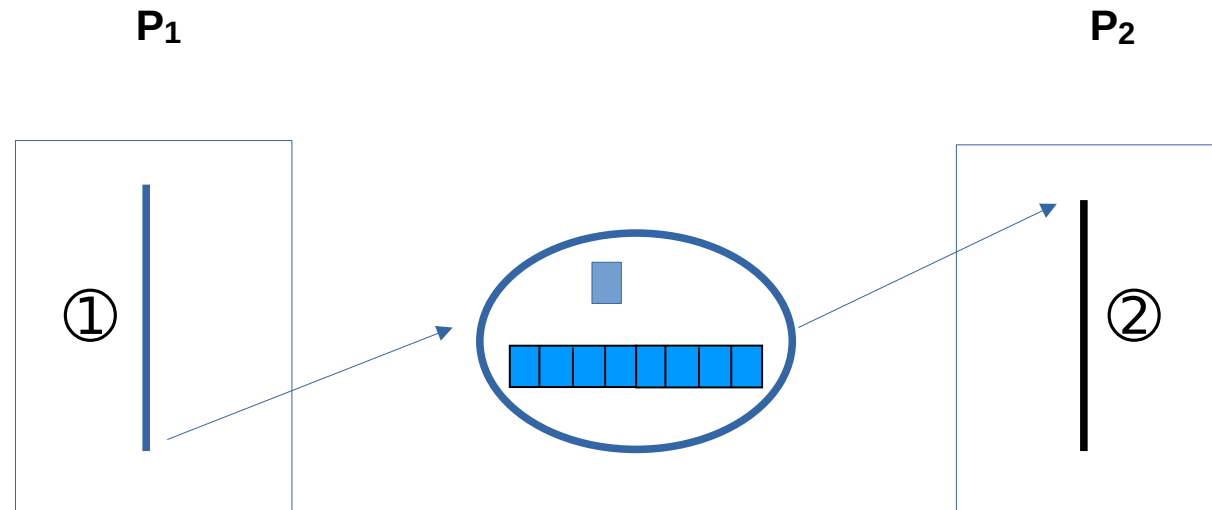
- Problème de la synchronisation relative de l'exécution des processus
= **dépendances fonctionnelles**



- P₂ ne doit pas appeler **RelevéCapteur()** tant que P₁ n'a pas effectué **InitCapteur()** !

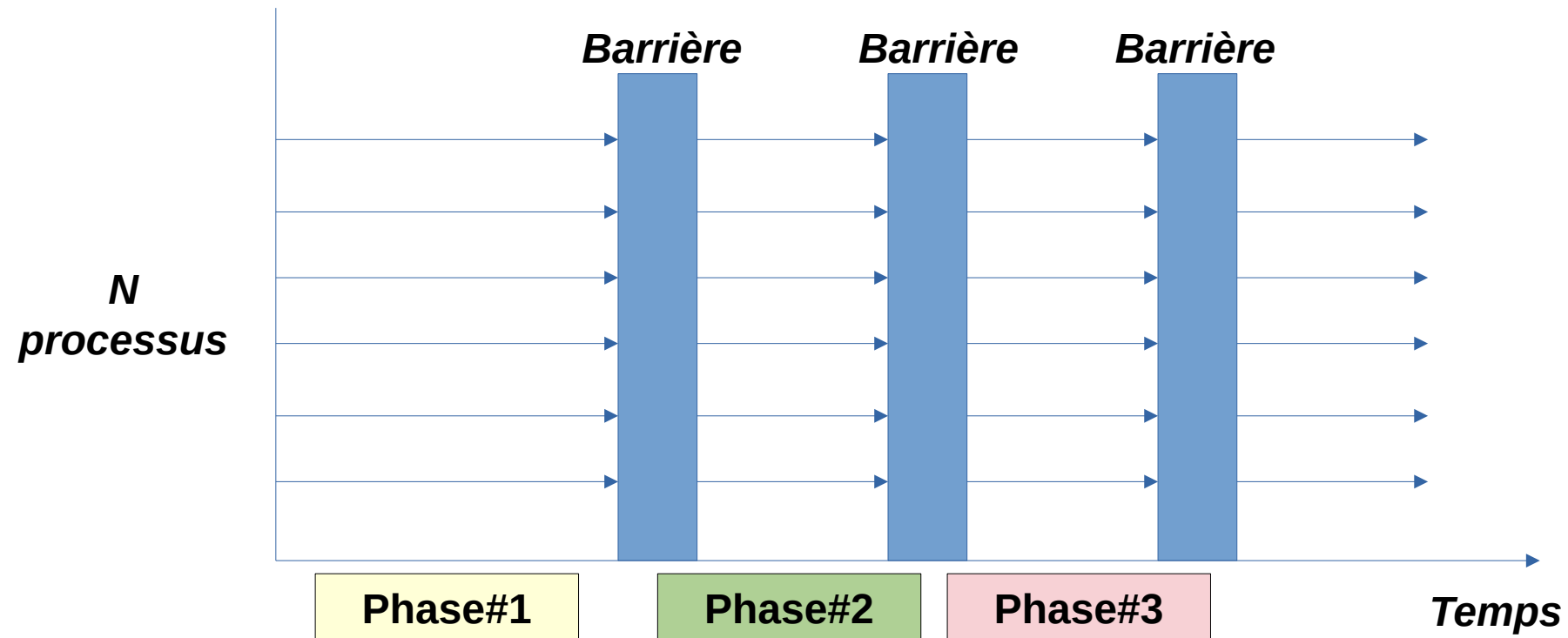
LES SEMAPHORES DE SYNCHRONISATION

- Mise en œuvre de **barrières de synchronisation**



LES SEMAPHORES

- Mise en œuvre de **rendez-vous**



FONCTIONS DE GESTION DES SEMAPHORES

- **Sémaphores anonymes POSIX** (librairie C <semaphore.h>) :

`int sem_init(sem_t *sem, int pshared, unsigned int value);`
initialisation d'un sémaphore

`int sem_wait(sem_t *sem);`
test de prise d'un sémaphore (sem.P())

`int sem_post(sem_t *sem);`
libération d'un sémaphore (sem.V())

`int sem_getvalue(sem_t *sem, int *valp);`
récupération de la valeur courante d'un sémaphore

`int sem_destroy(sem_t *sem);`
destruction d'un sémaphore

EXAMPLE

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem;

void *codeThreadDebutPhrase(void *arg) {
    printf("Bonjour\n");
    sem_post(&sem);
    pthread_exit(NULL);
}

void *codeThreadFinPhrase(void *arg) {
    sem_wait(&sem);
    printf("tout le monde !\n ");
    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t ThreadDebutPhrase, ThreadFinPhrase;
    sem_init(&sem, 0, 0);
    pthread_create(&ThreadFinPhrase, NULL,
                  codeThreadFinPhrase, NULL);

    sleep(2);
    pthread_create(&ThreadDebutPhrase, NULL,
                  codeThreadDebutPhrase, NULL);

    pthread_join(ThreadDebutPhrase, NULL);
    pthread_join(ThreadFinPhrase, NULL);
    sem_destroy(&sem);
    return 0;
}
```

```
Bonjour
tout le monde !
```

FIN